

IDATT2503 Assignment 10 Crypyo 04

Problem 1

1a

Alice should choose $q = 2027$ because it ensures that d is coprime with $\phi(n)$ and will provide better security by maximizing the difference between p and q .

An effective attack to factorize n for a weak choice, would be Fermat's Factorization. When p and q are close, i.e. if she chose $q = 1307$, Fermat's method can quickly factor n by exploiting the fact that $n = (a - b)(a + b)$ for some integers a and b .

1b

The Extended Euclidean algorithm involves iterative division and back-substitution steps to find the modular inverse.

```
# 1b

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        gcd, x1, y1 = extended_gcd(b % a, a)
        # Update x and y using the results of recursion
        x = y1 - (b // a) * x1
        y = x1
        return (gcd, x, y)

def modinv(a, m):
    """
    Computes the modular inverse of a modulo m
    """
    gcd, x, y = extended_gcd(a, m)
    if gcd != 1:
        raise Exception(f"Modular inverse does not exist for a = {a} and modulus m = {m}")
    else:
        return x % m

# Given values
p = 1283
q = 2027
d = 3

# Compute n and phi(n)
n = p * q
phi_n = (p - 1) * (q - 1)
```

```

# Compute e such that e * d ≡ 1 mod phi(n)
try:
    e = modinv(d, phi_n)
    print(f"Public exponent e is: {e}")
except Exception as ex:
    print(ex)

# Verify that (e * d) % phi_n == 1
verification = (e * d) % phi_n
print(f"Verification (e * d) mod phi(n) = {verification}")

```

```

Public exponent e is: 1731555
Verification (e * d) mod phi(n) = 1

```

1c

Encrypting the message 111 using Repeated Squaring.

```

# 1c

def modular_exponentiation(base, exponent, modulus):
    """
    Performs modular exponentiation using the repeated squaring method.
    Computes (base ** exponent) % modulus efficiently.
    """
    result = 1
    base = base % modulus # Ensure base is within modulus

    while exponent > 0:
        # If exponent is odd, multiply the result by the current base
        if exponent % 2 == 1:
            result = (result * base) % modulus
        # Square the base
        base = (base * base) % modulus
        # Halve the exponent (integer division)
        exponent = exponent // 2

    return result

# Given values
m = 111 # Message to encrypt
e = 1731555 # Public exponent
n = 2600641 # Modulus

# Encrypt the message
ciphertext = modular_exponentiation(m, e, n)
print(f"The encrypted message (ciphertext) is: {ciphertext}")

# Decryption using the private key
d = 3 # Private exponent
decrypted_message = modular_exponentiation(ciphertext, d, n)
print(f"The decrypted message is: {decrypted_message}")

```

The encrypted message (ciphertext) is: 1509208

The decrypted message is: 111
